

PART-A

1. A robot needs to find how far it must travel between two points on a 2D plane. Develop a C program to calculate the straight-line distance between the given coordinates.

Program

```
#include <stdio.h>
#include <math.h>
int main() {
    float x1, y1, x2, y2, d;
    printf("Enter x1 y1: ");
    scanf("%f %f", &x1, &y1);
    printf("Enter x2 y2: ");
    scanf("%f %f", &x2, &y2);
    d = sqrt((x2 - x1)*(x2 - x1) + (y2 - y1)*(y2 - y1));
    printf("Distance = %.2f", d);
    return 0;
}
```

Algorithm

1. Start
2. Declare variables: x1, y1, x2, y2, and d as float.
3. Prompt user to enter the coordinates of the first point (x1, y1).
4. Read input values for x1 and y1.
5. Prompt user to enter the coordinates of the second point (x2, y2).
6. Read input values for x2 and y2.
7. Compute the distance using the formula:
$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$
8. Display the result as Distance = d.
9. Stop

2. Develop a C program that takes a student's marks as input and displays their grade based on the following criteria: 90 and above: Grade A 75 to 89: Grade B 60 to 74: Grade C 50 to 59: Grade D Below 50: Grade F Choose a suitable control structure to implement this logic efficiently.

Program

```
#include <stdio.h>
int main() {
    int marks;
    printf("Enter marks: ");
```

```

scanf("%d", &marks);
if (marks >= 90)
    printf("Grade A");
else if (marks >= 75)
    printf("Grade B");
else if (marks >= 60)
    printf("Grade C");
else if (marks >= 50)
    printf("Grade D");
else
    printf("Grade F");
return 0;
}

```

Algorithm

1. Start
2. Declare an integer variable marks.
3. Prompt the user to enter marks.
4. Read the value of marks.
5. Check conditions to determine grade:
 - If marks >= 90, display "Grade A".
 - Else if marks >= 75, display "Grade B".
 - Else if marks >= 60, display "Grade C".
 - Else if marks >= 50, display "Grade D".
 - Else, display "Grade F".
6. Stop

3. **Develop a C program that takes a unique identification input like PAN Number, AADHAR_Number, APAAR_Id, Driving License, Passport and checks it against a set of stored KYC records. Based on the input, display whether the individual is verified or not. Use an appropriate control structure to handle multiple possible ID matches. Assume all Unique identification are of integer type.**

Program

```

#include <stdio.h>
int main() {
    int kyc[5] = {1111, 2222, 3333, 4444, 5555}; // PAN, AADHAR, APAAR, DL,
    Passport
    int id, i, found = 0;
    printf("Enter Unique ID: ");
}

```

```

scanf("%d", &id);
for (i = 0; i < 5; i++) {
    if (id == kyc[i]) {
        found = 1;
        break;
    }
}
if (found)
    printf("Individual is VERIFIED");
else
    printf("Individual is NOT VERIFIED");
return 0;
}

```

Algorithm

1. Start
2. Declare an integer array kyc[5] = {1111, 2222, 3333, 4444, 5555} to store valid IDs.
3. Declare integer variables id, i, and found = 0.
4. Prompt the user to enter a unique ID.
5. Read the value of id.
6. Repeat the following steps for each element in the kyc array (from i = 0 to i < 5):
 - If id is equal to kyc[i]:
 - Set found = 1.
 - Exit the loop.
7. Check the value of found:
 - If found == 1, display "Individual is VERIFIED".
 - Else, display "Individual is NOT VERIFIED".
8. Stop

4. **A math app needs to determine the type of roots for a quadratic equation based on user input. Develop a C program to calculate and display the roots based on the given coefficients.**

Program

```

#include <stdio.h>
#include <math.h>
int main() {
    float a, b, c, d, r1, r2;
    printf("Enter coefficients a, b and c: ");
    scanf("%f %f %f", &a, &b, &c);
    d = b*b - 4*a*c;
}

```

```

if (d > 0) {
    r1 = (-b + sqrt(d)) / (2*a);
    r2 = (-b - sqrt(d)) / (2*a);
    printf("Roots are real and distinct\n");
    printf("Root1 = %.2f\nRoot2 = %.2f", r1, r2);
}
else if (d == 0) {
    r1 = -b / (2*a);
    printf("Roots are real and equal\n");
    printf("Root = %.2f", r1);
}
else {
    printf("Roots are imaginary");
}
return 0;
}

```

Algorithm

1. Start
2. Declare variables a, b, c, d, r1, and r2 as float.
3. Prompt the user to enter the coefficients a, b, and c.
4. Read the values of a, b, and c.
5. Compute the discriminant using the formula:

$$d = b^2 - 4ac$$

6. Check the value of the discriminant (d):
 - If $d > 0$:
 - Calculate $r1 = \frac{-b + \sqrt{d}}{2a}$
 - Calculate $r2 = \frac{-b - \sqrt{d}}{2a}$
 - Display: "Roots are real and distinct"
 - Display: both roots r1 and r2
 - Else if $d == 0$:
 - Calculate $r1 = \frac{-b}{2a}$
 - Display: "Roots are real and equal"
 - Display: the root r1
 - Else:
 - Display: "Roots are imaginary"

7. Stop

5. A sensor in a robotic arm needs to calculate the angle of rotation in real-time, but the hardware doesn't support built-in trigonometric functions. Develop a C program to approximate the value of $\sin(x)$ using a series expansion method for improved performance.

Program

```
#include <stdio.h>

int main() {
    float x, term, sum;
    int n, i;
    printf("Enter value of x (in radians): ");
    scanf("%f", &x);
    term = x;
    sum = x;
    for (i = 1; i <= 5; i++) {
        term = -term * x * x / ((2*i) * (2*i + 1));
        sum = sum + term;
    }
    printf("Approximate value of sin(x) = %.4f", sum);
    return 0;
}
```

Algorithm

1. Start
2. Declare variables: x, term, sum as float and n, i as int.
3. Display message: "Enter value of x (in radians):".
4. Read the value of x.
5. Set term = x.
6. Set sum = x.
7. Repeat the following steps for i from 1 to 5:
 - Update term using
$$\text{term} = -\text{term} \times x \times x \div ((2 \times i) \times (2 \times i + 1))$$
 - Add term to sum.
8. Display "Approximate value of $\sin(x)$ = " followed by the value of sum.
9. Stop.

6. Develop a C program that accepts a course description string and a keyword from the user. Search whether the keyword exists within the course description using appropriate string functions. If found, display: "Keyword " found in the course description." Otherwise, display: "Keyword " not found in the course description."

Program

```

#include <stdio.h>
#include <string.h>
int main() {
    char desc[200], key[50];
    printf("Enter course description: ");
    gets(desc);
    printf("Enter keyword to search: ");
    gets(key);
    if (strstr(desc, key) != NULL)
        printf("Keyword '%s' found in the course description.", key);
    else
        printf("Keyword '%s' not found in the course description.", key);
    return 0;
}

```

Algorithm

1. Start
2. Declare two character arrays:
 - desc[200] for storing the course description.
 - key[50] for storing the keyword to be searched.
3. Prompt the user to enter the course description.
4. Read the course description into desc.
5. Prompt the user to enter a keyword to search.
6. Read the keyword into key.
7. Search for the keyword in the description using strstr(desc, key).
8. If the keyword is found (strstr returns a non-NULL value):
 - Display: “Keyword '<key>' found in the course description.”
9. Else:
 - Display: “Keyword '<key>' not found in the course description.”
10. Stop

7. **Develop a C program that takes marks for three subjects as input. Use a function to check if the student has passed (minimum 40 marks in each subject). Display the average and whether the student passed or failed.**

Program

```

#include <stdio.h>
int checkPass(int m1, int m2, int m3) {
    if (m1 >= 40 && m2 >= 40 && m3 >= 40)
        return 1;
    else
        return 0;
}

```

```

}
int main() {
    int m1, m2, m3;
    float avg;
    printf("Enter marks of three subjects: ");
    scanf("%d %d %d", &m1, &m2, &m3);
    avg = (m1 + m2 + m3) / 3.0;
    if (checkPass(m1, m2, m3))
        printf("Average = %.2f\nResult: PASS", avg);
    else
        printf("Average = %.2f\nResult: FAIL", avg);
    return 0;
}

```

Algorithm

1. Start
2. Declare integer variables m1, m2, and m3, and float variable avg.
3. Prompt the user to enter marks of three subjects.
4. Read the values of m1, m2, and m3.
5. Calculate the average using the formula:

$$avg = (m1 + m2 + m3) / 3.0$$
6. Call the function checkPass(m1, m2, m3).
7. Inside the function checkPass:
 - If all marks are greater than or equal to 40 ($m1 \geq 40 \ \&\& \ m2 \geq 40 \ \&\& \ m3 \geq 40$),
return 1.
 - Else, return 0.
8. Back in main function:
 - If the returned value is 1, display:
 - "Average = (value of avg)"
 - "Result: PASS"
 - Else, display:
 - "Average = (value of avg)"
 - "Result: FAIL"
9. Stop

8. In an ATM system, two account balances need to be swapped temporarily for validation. Develop a C program that accepts two balances and uses a function with pointers to swap them. Display the balances before and after swapping.

Program

```
#include <stdio.h>

void swap(float *a, float *b) {
    float temp;
    temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    float bal1, bal2;
    printf("Enter two account balances: ");
    scanf("%f %f", &bal1, &bal2);
    printf("Before swapping:\n");
    printf("Balance 1 = %.2f\nBalance 2 = %.2f\n", bal1, bal2);
    swap(&bal1, &bal2);
    printf("After swapping:\n");
    printf("Balance 1 = %.2f\nBalance 2 = %.2f\n", bal1, bal2);
    return 0;
}
```

Algorithm

1. Start
2. Declare float variables bal1 and bal2.
3. Prompt the user to enter two account balances.
4. Read the values of bal1 and bal2.
5. Display the balances before swapping:
 - "Balance 1 = (value of bal1)"
 - "Balance 2 = (value of bal2)"
6. Call the function swap(&bal1, &bal2) passing addresses of bal1 and bal2.
7. Inside the function swap:
 - Declare a temporary variable temp.
 - Store the value at address a in temp (temp = *a).
 - Copy the value at address b to address a (*a = *b).
 - Copy the value from temp to address b (*b = temp).
 - Return to main function.
8. Back in main function:
 - Display the balances after swapping:
 - "Balance 1 = (value of bal1)"

- "Balance 2 = (value of bal2)"

9. Stop

PART-B

1. **A college library has a digital bookshelf system where each book is assigned a unique Book ID. The bookshelf is organized in ascending order of Book IDs. Develop a C Program to quickly find whether a book with a specific Book ID is available in the shelf.**

Program

```
#include <stdio.h>
int main() {
    int a[50], n, key, low, high, mid, i;
    printf("Enter number of books: ");
    scanf("%d", &n);
    printf("Enter Book IDs in ascending order:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter Book ID to search: ");
    scanf("%d", &key);
    low = 0;
    high = n - 1;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) {
            printf("Book ID %d is AVAILABLE in the shelf", key);
            return 0;
        }
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    printf("Book ID %d is NOT AVAILABLE in the shelf", key);
    return 0;
}
```

Algorithm

1. Start
2. Declare an integer array a[50] and variables n, key, low, high, mid, and i.

3. Prompt the user to enter the number of books.
4. Read the value of n.
5. Prompt the user to enter book IDs in ascending order.
6. Read the book IDs into array a[].
7. Prompt the user to enter the Book ID to search (store it in key).
8. Initialize low = 0 and high = n - 1.
9. Repeat the following steps while low <= high:
 - Calculate mid = (low + high) / 2.
 - If a[mid] == key:
 - Display: "Book ID (key) is AVAILABLE in the shelf."
 - Stop.
 - Else if key < a[mid]:
 - Update high = mid - 1.
 - Else:
 - Update low = mid + 1.
10. If the loop ends without finding the key:
 - Display: "Book ID (key) is NOT AVAILABLE in the shelf."
11. Stop

2. A sports teacher has recorded the scores of students in a 100-meter race. To prepare the result sheet, the teacher wants the scores arranged in descending order (from highest to lowest). Develop a C program to sort the scores.

Program

```
#include <stdio.h>

int main() {
    int a[20], n, i, j, temp;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (a[i] < a[j]) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
}
```

```

printf("Scores in descending order:\n");
for (i = 0; i < n; i++)
    printf("%d ", a[i]);
return 0;
}

```

Algorithm

1. Start
2. Declare an integer array a[20] and variables n, i, j, and temp.
3. Read the number of scores, n.
4. Read n scores into the array a[].
5. Repeat for each element of the array (i = 0 to n - 2):
 - For each subsequent element (j = i + 1 to n - 1):
 - If a[i] < a[j], then:
 - Swap a[i] and a[j] using a temporary variable temp.
6. After all comparisons, the array a[] will be sorted in descending order.
7. Display the message: "Scores in descending order:"
8. Print all elements of the sorted array a[].
9. Stop

3. **A small warehouse tracks how many units of different products are shipped from multiple branches. Another dataset shows how much revenue each product generates per unit. Develop a C program which combines these datasets to calculate the total revenue generated by each branch.**

Program

```

#include <stdio.h>
int main() {
    int units[10][10], revenue[10];
    int b, p, i, j, total;
    printf("Enter number of branches and products: ");
    scanf("%d %d", &b, &p);
    printf("Enter units shipped (branch-wise):\n");
    for (i = 0; i < b; i++)
        for (j = 0; j < p; j++)
            scanf("%d", &units[i][j]);
    printf("Enter revenue per unit for each product:\n");
    for (j = 0; j < p; j++)
        scanf("%d", &revenue[j]);
    for (i = 0; i < b; i++) {
        total = 0;

```

```

        for (j = 0; j < p; j++)
            total += units[i][j] * revenue[j];
        printf("Total revenue of Branch %d = %d\n", i + 1, total);
    }
    return 0;
}

```

Algorithm

1. Start
2. Declare a 2D integer array `units[10][10]`, and 1D integer array `revenue[10]`.
3. Declare integer variables: `b`, `p`, `i`, `j`, and `total`.
4. Prompt the user to enter the number of branches (`b`) and products (`p`).
5. Read the values of `b` and `p`.
6. Prompt the user to enter the number of units shipped by each branch for each product.
7. Store the input values into the 2D array `units[i][j]` using nested loops:
 - Outer loop (for each branch `i` from 0 to `b - 1`)
 - Inner loop (for each product `j` from 0 to `p - 1`)
8. Prompt the user to enter the revenue per unit for each product.
9. Read and store these values in the `revenue[]` array.
10. For each branch (loop `i = 0` to `b - 1`):
 - Set `total = 0`.
 - For each product `j = 0` to `p - 1`, calculate and add to `total`:

$$total = total + (units[i][j] \times revenue[j])$$
 - Display: "Total revenue of Branch (`i + 1`) = `total`".
11. Stop

4. **A basic mobile contact manager stores first and last names separately. For displaying full names in the contact list, you need to join them manually. Additionally, the system must check the length of each full name to ensure it fits the screen. Perform these operations by developing a C program without using builtin string functions.**

Program

```

#include <stdio.h>
int main() {
    char first[20], last[20], full[40];
    int i = 0, j = 0, len = 0;
    printf("Enter first name: ");
    scanf("%s", first);
}

```

```

printf("Enter last name: ");
scanf("%s", last);
// Copy first name
while (first[i] != '\0') {
    full[j++] = first[i++];
}
full[j++] = ' '; // space between names
i = 0;
//Copy last name
while (last[i] != '\0') {
    full[j++] = last[i++];
}
full[j] = '\0';
// Calculate length
i = 0;
while (full[i] != '\0') {
    len++;
    i++;
}
printf("Full Name: %s\n", full);
printf("Length of Full Name: %d\n", len);
if (len <= 20)
    printf("Name fits the screen");
else
    printf("Name does NOT fit the screen");
return 0;
}

```

Algorithm

1. Start
2. Declare three character arrays:
 - first[20] for first name.
 - last[20] for last name.
 - full[40] for the full name (first + last).
3. Declare integer variables: i = 0, j = 0, and len = 0.
4. Prompt the user to enter the first name.
5. Read the first name into first.
6. Prompt the user to enter the last name.
7. Read the last name into last.
8. Copy characters from first into full one by one until the null character ('\0') is reached.
9. Add a space character to full after the first name.

10. Reset $i = 0$.
11. Copy characters from last into full one by one until the null character is reached.
12. Add the null terminator ('\0') at the end of the full string.
13. Initialize $len = 0$ and count the number of characters in full until the null character to find its length.
14. Display the full name and its length.
15. If $len \leq 20$, then display: "Name fits the screen."
16. Else, display: "Name does NOT fit the screen."
17. Stop

- 5. A currency exchange booth allows users to convert between two currencies. Before confirming the exchange, the system simulates a swap of the values to preview the result without actually changing the original data. In other cases, it updates the actual values. Develop a C program that implements both behaviours using Call by Value and Call by reference**

Program

```
#include <stdio.h>
//Call by Value: Simulation only
void simulateSwap(float a, float b) {
    float temp;
    temp = a;
    a = b;
    b = temp;
    printf("After simulation (Call by Value): %.2f %.2f\n", a, b);
}
// Call by Reference: Actual swap
void actualSwap(float *a, float *b) {
    float temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    float c1, c2;
    printf("Enter two currency values: ");
    scanf("%f %f", &c1, &c2);
    printf("Before swap: %.2f %.2f\n", c1, c2);
    simulateSwap(c1, c2); // values unchanged
    actualSwap(&c1, &c2); // values changed
```

```

    printf("After actual swap (Call by Reference): %.2f %.2f\n", c1, c2);
    return 0;
}

```

Algorithm

1. Start
2. Declare float variables c1 and c2.
3. Prompt the user to enter two currency values.
4. Read the values of c1 and c2.
5. Display the message: "Before swap" along with the original values of c1 and c2.
6. Call the function simulateSwap(c1, c2) to perform a *simulated swap (Call by Value)*.

Inside simulateSwap(a, b)

- Declare float variable temp.
 - Store the value of a in temp.
 - Assign the value of b to a.
 - Assign the value of temp to b.
 - Display the swapped values of a and b.
 - Return to main. (*Note: original values in main remain unchanged.*)
7. Call the function actualSwap(&c1, &c2) to perform an *actual swap (Call by Reference)*.

Inside actualSwap(*a, *b)

- Declare float variable temp.
 - Store the value pointed by a in temp.
 - Assign the value pointed by b to *a.
 - Assign the value in temp to *b.
 - Return to main. (*Original variables are now swapped.*)
8. Display the message: "After actual swap (Call by Reference)" with the updated values of c1 and c2.
 9. Stop

6. **A local library needs to store and display details of its books, including title, author, and year of publication. Design a structure that can hold these details and develop a C program to display a list of all books entered.**

Program

```

#include <stdio.h>

struct Book {
    char title[50];
    char author[50];

```

```

    int year;
};
int main() {
    struct Book b[10];
    int n, i;
    printf("Enter number of books: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter title, author and year:\n");
        scanf("%s %s %d", b[i].title, b[i].author, &b[i].year);
    }
    printf("\nBook List:\n");
    for (i = 0; i < n; i++) {
        printf("%s %s %d\n", b[i].title, b[i].author, b[i].year);
    }
    return 0;
}

```

Algorithm

1. Start
2. Define a structure named Book with the following members:
 - title (character array of size 50)
 - author (character array of size 50)
 - year (integer for publication year)
3. Declare an array b[10] of type struct Book to store details of up to 10 books.
4. Declare integer variables n and i.
5. Prompt the user to enter the number of books (n).
6. Read the value of n.
7. Repeat the following steps for each book i from 0 to n - 1:
 - Prompt: "Enter title, author and year:"
 - Read and store each book's title, author, and year in b[i].
8. Display the message: "Book List:".
9. Repeat for each book i from 0 to n - 1:
 - Print the book's title, author, and year.
10. Stop